



WEB APPLICATION DEVELOPMENT-1 MID II QUESTION BANK

SHORT ANSWER TYPE QUESTIONS:

1. What is a CSS transition?

Ans: CSS transition enables gradual change of CSS properties from one value to another over time.

Example: Changing background color, size, opacity, position, etc., smoothly when triggered by events such as **hover**, **focus**, or **active**.

2. What is CSS animation?

Ans: CSS animation is a feature in Cascading Style Sheets (CSS) that allows you to animate HTML elements by smoothly transitioning their CSS properties over time. This enables the creation of dynamic and interactive visual effects on web pages without the need for JavaScript in many cases.

@keyframes Rule: This rule defines the different stages of an animation.

- animation-name: Specifies the name of the @keyframes rule to be used (e.g., myAnimation).
- animation-duration: Defines how long the animation should take to complete (e.g., 4s)

3. What is the difference between transition and animation?

Transition	Animations
Transitions cannot loop (You can make them do that but they are not designed for that).	Animations have no problem in looping.
Transitions need a trigger to run like mouse hover.	The animation just start. They don't need any kind of external trigger source.
Transitions are easy to work in JavaScript.	The animations are hard to work in JavaScript. The syntax for manipulating a keyframe and assigning a new value to it, is very complex.

Transitions animate a object from one point to another.	Animation allows you to define Keyframes which varies from one state to another with various properties and time frame.
Use transition for manipulating the value using JavaScript.	Flexibility is provided by having multiple keyframes and easy loop.

4. What is parameter passing

Parameter passing is the process of providing inputs (arguments) to a function through its parameters, allowing the function to work with those values.

Example:

```
function add(x, y) { // x and y are parameters
  return x + y; }
```

5. Define innerText and innerHTML.

Ans: **innerText** is a property in JavaScript used to get or set **the visible text content** of an element.

Key Points:

- It returns **only the text that is visible** (not hidden by CSS).
- Ignores HTML tags.
- Triggered by layout changes (reflow), so it is slower.

innerHTML is a property in JavaScript used to get or set **the HTML content** inside an element.

Key Points:

- Returns both **text + HTML tags**.
- Allows you to insert HTML elements.
- Faster because it does not consider CSS visibility.

6. What is Conditional statement?

Ans: Conditional statements in JavaScript enable code to make decisions and execute different blocks of code based on whether a specified condition evaluates to true or false.

Example:

if statement: Executes a block of code only if a specified condition is true.

if...else statement: Executes one block of code if the condition is true, and a different block of code if the condition is false.

7. What is attribute manipulation? Give an example.

Attribute manipulation refers to changing, updating, adding, or removing the attributes of an HTML element using JavaScript.

Attributes include things like src, href, id, class, value, style, title, etc.

With attribute manipulation, we can dynamically modify the properties of elements in a webpage.

```
<p id="text">Hello World</p>
```

```
<script>
```

```
document.getElementById("text").setAttribute("style", "color: blue;");
```

```
</script>
```

8. What are logical operators? Give examples.

Logical operators are symbols that are used to connect two or more conditions and return a result in the form of true or false.

They help in decision-making during program execution.

Given that **x = 6 and y = 3**, the table below explains the logical operators:

Oper	Name	Example
&&	AND	(x < 10 && y > 1) is true
	OR	(x === 5 y === 5) is false
!	NOT	!(x === y) is true

9. Write any two element selection methods in DOM.

Ans: document.getElementById(): This method selects a single element based on its unique id attribute. Since id attributes are meant to be unique within an HTML document, this method is efficient for directly targeting a specific element.

document.getElementsByClassName(): This method selects all elements that share a specified class name. It returns a live HTMLCollection (an array-like object) of all matching elements in the order they appear in the document

10. Define event propagation

Event propagation is the process that explains how an event travels through different levels of elements in a webpage when it is triggered. When we click an element, the event does not stop there—it moves through its parent and child elements in a specific order.

- Capturing Phase (Event Capture)
- Target Phase
- Bubbling Phase

11. Write syntax of Fetch API for a GET request.

Ans: `fetch('https://example.com/api/data')`

```
.then(response => response.json())
.then(data => {
  console.log(data);
})
.catch(error => {
  console.error('Error:', error);
});
```

12. What is form validation?

Ans: Form validation is the process of checking user input in a web form to ensure it meets specific criteria and requirements before it is submitted to a server or processed further. It is a crucial part of web development that aims to improve data quality, enhance user experience, and bolster security.

Key aspects of form validation include:

Ensuring completeness, Validating data format, Enforcing constraints, Maintaining consistency and Preventing malicious input.

13.What are JSON objects and JSON arrays?

A JSON object is a collection of data written in key -value pairs. It is enclosed within curly braces { } and each key is followed by a value using a colon ":"

Data in it can be number, string, boolean, null

```
{
  "name": "Alice",
  "age": 30,
  "isStudent": false,
  "address":
  {
    "street": "123 Main St",
    "city": "Anytown"
  }
}
```

JSON Array:

A JSON array represents an ordered collection of values. It is enclosed in square brackets []

Values: Can be any valid JSON data type: string, number, boolean, null, a JSON object, or another JSON array.

```
[
  "apple",
  "banana",
  "orange"
]
```

14.List any four input element types.

Ans: Here are four common input element types used in HTML forms:

- **Text:** Used for single-line text input.
- **Password:** Used for password input, masking the characters as they are typed.
- **Email:** Used for entering email addresses, often including validation for a valid email format.
- **Number:** Used for entering numerical values, often with built-in controls for incrementing/decrementing.

15. What is the difference between XML and JSON?

Ans:

JSON	XML
It is JavaScript Object Notation	It is Extensible markup language
It is based on JavaScript language.	It is derived from SGML.
It is a way of representing objects.	It is a markup language and uses tag structure to represent data items.
It does not provides any support for namespaces.	It supports namespaces.
It supports array.	It doesn't supports array.
Its files are very easy to read as compared to XML.	Its documents are comparatively difficult to read and interpret.
It doesn't use end tag.	It has start and end tags.
It is less secured.	It is more secured than JSON.
It doesn't supports comments.	It supports comments.
It supports only UTF-8 encoding.	It supports various encoding.

PART-2: LONG ANSWER TYPE QUESTIONS:

1. Explain the CSS Grid system in detail. Discuss grid container and grid item properties with examples.

CSS Grid is a powerful layout system used in web design to create rows and columns easily. It allows us to design complex web-page layouts in a structured manner

CSS Grid Components

A grid always consists of:

- **A Grid Container** - The parent (container) element, where the display property is set to grid or inline-grid
- **One or more Grid Items** - The direct children of the grid container automatically becomes grid items

All CSS Grid Properties

Property	Description
<u>align-content</u>	Vertically aligns the whole grid inside the container (when total grid size is smaller than container)
<u>align-items</u>	Specifies the default alignment for items inside a flexbox or grid container
<u>align-self</u>	Aligns the content for a specific grid item along the column axis
<u>display</u>	Specifies the display behavior (the type of rendering box) of an element
<u>column-gap</u>	Specifies the gap between the columns

CSS Grid Container

A grid container contains one or more grid items arranged in columns and rows.

All direct child elements of a grid container automatically become grid items.

An element becomes a grid container when its display property is set to grid or inline-grid.

Example

Use display: grid to make a block-level grid container:

```
.container {
  display: grid;
}
```

Example

Use display: inline-grid to make an inline grid container:

```
.container {  
  display: inline-grid;  
}
```

Result:



CSS align-content Property

The align-content property specifies how flex lines are distributed along the cross axis in a flexbox container.

In flexbox layout, the main axis is in the [flex-direction](#) (default is 'row', horizontal), and the cross axis is perpendicular to the main axis (default is 'column', vertical).

Example:

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
body {  
  background-color:#E7E9EB;  
}  
#main {  
  background-color:#FFFFFFF;  
  width: 100%;  
  height: 300px;  
  display: flex;  
  flex-wrap: wrap;  
  align-content: stretch;  
}  
  
#main div {
```

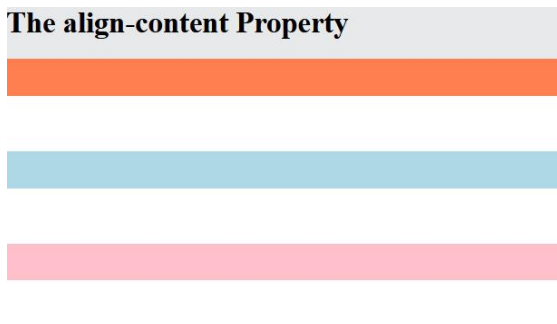
```
width: 100%;
height: 40px;
}
</style>
</head>
<body>

<h1>The align-content Property</h1>

<div id="main">
  <div style="background-color:coral;"></div>
  <div style="background-color:lightblue;"></div>
  <div style="background-color:pink;"></div>
</div>

</body>
</html>
```

OUTPUT:



2. Write short notes on Transition Timing Functions. Explain with syntax.

Transition timing functions are used in CSS to control how a transition progresses over time. Instead of changing the property at a constant rate, timing functions help us make the animation slow, fast, or smooth at different points.

Predefined Keywords:

- **ease (default):** Starts slowly, accelerates quickly, and then decelerates at the end.
- **linear:** Maintains a constant speed throughout the transition.
- **ease-in:** Starts slowly and then accelerates.
- **ease-out:** Starts quickly and then decelerates towards the end.

- **ease-in-out:** Starts and ends slowly, with a faster pace in the middle.
- **step-start:** Jumps instantly to the final state at the beginning of the transition.
- **step-end:** Stays at the initial state until the very end, then jumps instantly to the final state.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 50px;
  background: red;
  color: white;
  font-weight: bold;
  transition: width 2s;
}

#div1 {transition-timing-function: linear;}
#div2 {transition-timing-function: ease;}
#div3 {transition-timing-function: ease-in;}
#div4 {transition-timing-function: ease-out;}
#div5 {transition-timing-function: ease-in-out;}

div:hover {
  width: 300px;
}
</style>
</head>
<body>

<h1>The transition-timing-function Property</h1>

<div id="div1">linear</div>
<div id="div2">ease</div>
```

```
<div id="div3">ease-in</div>
<div id="div4">ease-out</div>
<div id="div5">ease-in-out</div>
```

<p>Hover over the div elements above, to see the different transition effects.</p>

```
</body>
</html>
```

OUTPUT:

The transition-timing-function Property



Hover over the div elements above, to see the different transition effects.

3. Explain briefly about animation timing functions.

Ans: Animation timing functions, also known as easing functions or speed curves, control the pace at which an animation progresses over its duration. They define how the animation's speed changes from its start to its end, influencing the "feel" of the movement. Instead of a constant, linear speed, timing functions allow for more dynamic and natural-looking animations by introducing acceleration and deceleration. This is achieved by mapping the animation's progress over time using various mathematical curves.

Value	Description
linear	The animation has the same speed from start to end
ease	Default value. The animation has a slow start, then fast, before it ends slowly
ease-in	The animation has a slow start
ease-out	The animation has a slow end

ease-in-out	The animation has both a slow start and a slow end
step-start	Equivalent to steps(1, start)
step-end	Equivalent to steps(1, end)
steps(int,step-position)	Specifies a stepping function, with two parameters. The first parameter specifies the number of steps/intervals. The second parameter, specifies when the jump between values occurs
cubic-bezier(<i>n,n,n,n</i>)	Define your own values in the cubic-bezier function Possible values are numeric values from 0 to 1
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

Example:

```

<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 50px;
  background: red;
  color: white;
  font-weight: bold;
  position: relative;
  animation: mymove 5s infinite;
}

#div1 {animation-timing-function: linear;}
#div2 {animation-timing-function: ease;}
#div3 {animation-timing-function: ease-in;}
#div4 {animation-timing-function: ease-out;}
#div5 {animation-timing-function: ease-in-out;}

@keyframes mymove {
  from {left: 0px;}

```

```
to {left: 300px;}
}
</style>
</head>
<body>

<h1>Elements with Different timing function Values</h1>

<div id="div1">linear</div>
<div id="div2">ease</div>
<div id="div3">ease-in</div>
<div id="div4">ease-out</div>
<div id="div5">ease-in-out</div>

</body>
</html>
```

OUTPUT:

Elements with Different timing function Values



4. Explain javascript data types with example

JavaScript data types represent the type of value stored in a variable. They are mainly divided into two categories: Primitive and Non- Primitive

Type	Description
String	A text of characters enclosed in quotes
Number	A number representing a mathematical value
Bigint	A number representing a large integer
Boolean	A data type representing true or false
Object	A collection of key-value pairs of data
Undefined	A primitive variable with no assigned value
Null	A primitive value representing object absence
Symbol	A unique and primitive identifier

Examples

// Strings

```
let color = "Yellow";
```

```
let lastName = "Johnson";
```

// Number

```
let length = 16;
```

```
let weight = 7.5;
```

// BigInt

```
let x = 1234567890123456789012345n;
```

```
let y = BigInt(1234567890123456789012345)
```

// Boolean

```
let x = true;
```

```
let y = false;
```

// Object

```
const person = {firstName:"John", lastName:"Doe"};
```

// Array object

```
const cars = ["Saab", "Volvo", "BMW"];
```

// Date object

```
const date = new Date("2022-03-25");
```

```
// Undefined
```

```
let x;
```

```
let y;
```

```
// Null
```

```
let x = null;
```

```
let y = null;
```

```
// Symbol
```

```
const x = Symbol();
```

```
const y = Symbol();
```

JavaScript Types are Dynamic

JavaScript has dynamic types. This means that the same variable can be used to hold different data types:

Example

```
let x;    // Now x is undefined
```

```
x = 5;    // Now x is a Number
```

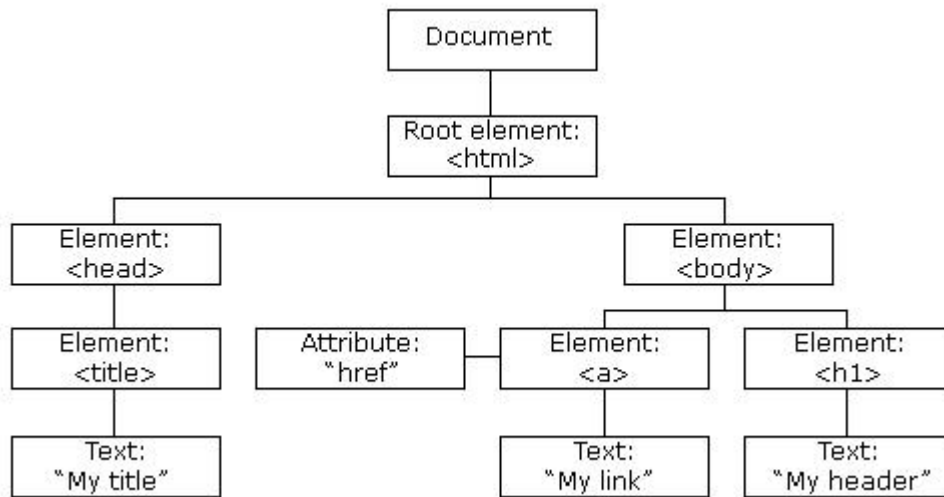
```
x = "John"; // Now x is a String
```

5. Explain DOM hierarchy in detail. Describe parent, child, and sibling relationships with examples.

DOM (Document Object Model) represents an HTML document in a tree structure.

In this tree, each HTML element is treated as a node, and nodes are connected in a family-like hierarchy.

- The entire document is a document node
- Every HTML element is an element node
- The text inside HTML elements are text nodes
- Every HTML attribute is an attribute node (deprecated)
- All comments are comment nodes



Node Relationships

The nodes in the node tree have a hierarchical relationship to each other.

The terms parent, child, and sibling are used to describe the relationships.

- In a node tree, the top node is called the root (or root node)
- Every node has exactly one parent, except the root (which has no parent)
- A node can have a number of children
- Siblings (brothers or sisters) are nodes with the same parent
- `<html>`

`<head>`

`<title>DOM Tutorial</title>`

`</head>`

`<body>`

`<h1>DOM Lesson one</h1>`

`<p>Hello world!</p>`

`</body>`

`</html>`

From the HTML above you can read:

- `<html>` is the root node

- <html> has no parents
- <html> is the parent of <head> and <body>
- <head> is the first child of <html>
- <body> is the last child of <html>

and:

- <head> has one child: <title>
- <title> has one child (a text node): "DOM Tutorial"
- <body> has two children: <h1> and <p>
- <h1> has one child: "DOM Lesson one"
- <p> has one child: "Hello world!"
- <h1> and <p> are siblings

6. Explain different types of loops in JavaScript with examples.

Ans : JavaScript loops **repeat executing a block of code** a number of times.

Loops are fundamental for tasks involving:

- Iteration over values
- Iteration over data structures
- Performing an action multiple times

Loop Type	Description
for	Iterates over values and expressions
while	Iterates over a condition
do...while	Iterates over a condition
for...in	Iterates over the properties of an Object
for...of	Iterates over array like objects
forEach()	Iterates over each element in an Array

Example; for loop

```
<!DOCTYPE html>
<html>
<body>
```

```
<h1>JavaScript Loops</h1>
```

```
<h2>The for Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = "";
```

```
for (let i = 0; i < 5; i++) {  
  text += "The number is " + i + "<br>";  
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT:

JavaScript Loops

The for Loop

```
The number is 0  
The number is 1  
The number is 2  
The number is 3  
The number is 4
```

Example: while loop

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Loops</h1>
```

```
<h2>The While Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = "";
```

```
let i = 0;
```

```
while (i < 10) {
```

```
  text += "The number is " + i + "<br>";
```

```
  i++;
```

```
}
```

```
document.getElementById("demo").innerHTML = text;
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT:

JavaScript Loops

The While Loop

```
The number is 0  
The number is 1  
The number is 2  
The number is 3  
The number is 4  
The number is 5  
The number is 6  
The number is 7  
The number is 8  
The number is 9
```

The for...in Loop

The for...in loop iterates over the enumerable properties of an object.

It is typically used for iterating over object keys.

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Loops</h1>
```

```
<h2>The for in Loop</h2>
```

```
<p>A for in statement loops through the properties of a person object:</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
const person = {fname:"John", lname:"Doe", age:25};
```

```
let txt = "";
```

```
for (let x in person) {  
  txt += person[x] + " ";  
}
```

```
document.getElementById("demo").innerHTML = txt;
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT:

JavaScript Loops

The for in Loop

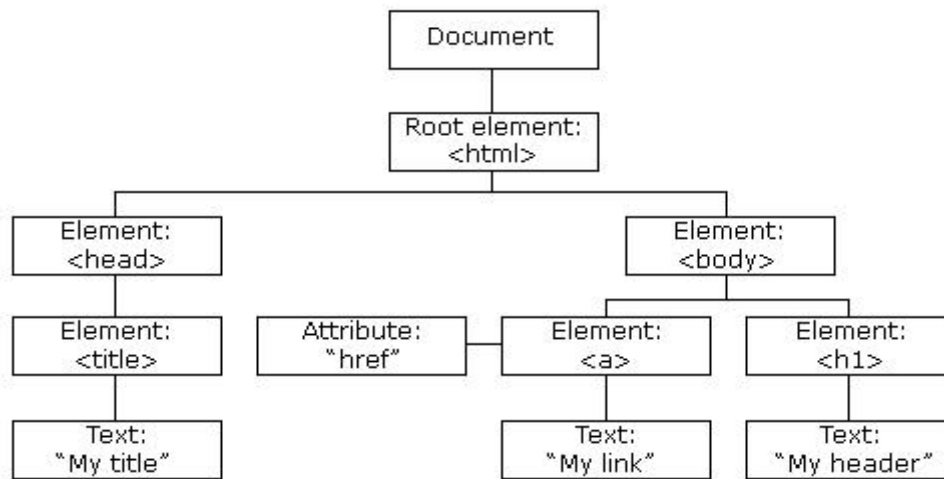
A for in statement loops through the properties of a person object:

John Doe 25

7. Explain the Document Object Model (DOM) in detail. Draw and explain the DOM tree structure.

Ans : When a web page is loaded, the browser creates a **Document Object Model** of the page. With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

The **HTML DOM** model is constructed as a tree of **Objects**:



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

Example:

<html>

<head>

<title>DOM Tutorial</title>

</head>

<body>

```
<h1>DOM Lesson one</h1>
<p>Hello world!</p>
</body>
```

```
</html>
```

From the HTML above you can read:

- <html> is the root node
- <html> has no parents
- <html> is the parent of <head> and <body>
- <head> is the first child of <html>
- <body> is the last child of <html>

and:

- <head> has one child: <title>
- <title> has one child (a text node): "DOM Tutorial"
- <body> has two children: <h1> and <p>
- <h1> has one child: "DOM Lesson one"
- <p> has one child: "Hello world!"
- <h1> and <p> are siblings

8. Explain different types of control structures in JavaScript.

Control structures are statements that control the flow of execution in a program. They decide which code should run, how many times it should run, and under what conditions.

JavaScript has three main types of control structures:

Conditional Control Flow

Conditions let you **make decisions** using:

- if
- if...else
- switch
- ternary (? :)

Example:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h1>JavaScript if Statement</h1>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let age = 18;
```

```
let text = "Unknown"
```

```
if (age >= 18) {
```

```
    text = "Adult";
```

```
} else {
```

```
    text = "Minor";
```

```
}
```

```
document.getElementById("demo").innerHTML = "The group is " + text;
```

```
</script>
```

```
</body>
```

```
</html>
```

Loops (Repetition Control Flow)

Loops let you **run code multiple times** using:

- for
- while
- do...while

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1>JavaScript Loops</h1>
```

```
<h2>The for Loop</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
let text = "";
```

```
for (let i = 0; i < 5; i++) {  
  text += "The number is " + i + "<br>";  
}  
  
document.getElementById("demo").innerHTML = text;  
</script>  
  
</body>  
</html>
```

Jump Statements

Jump statements let you **change the flow** abruptly using:

- break - exits a loop or switch
- continue - skips the current loop iteration
- return - exits from a function
- throw - jumps to error handling

Example:

```
<!DOCTYPE html>  
<html>  
<body>  
<h1>JavaScript break</h1>  
<h2>The break Statement in a Loop</h2>  
  
<p>Break the loop when the loop counter is 3:</p>  
  
<p id="demo"></p>  
  
<script>  
let text = "";  
  
for (let i = 0; i < 10; i++) {  
  if (i === 3) { break; }  
  text += "The number is " + i + "<br>";  
}  
  
document.getElementById("demo").innerHTML = text;
```

</script>

</body>

</html>

9. Explain node properties in JavaScript DOM. Provide suitable examples.

Ans : In JavaScript's Document Object Model (DOM), everything within an HTML document is represented as a node. These nodes possess various properties that allow for interaction and manipulation of the document's structure, content and style.

Common Node Properties:

nodeType: Returns a number representing the type of the node.

- 1 for Element nodes (e.g., <div>, <p>).
- 2 for Attribute nodes (e.g., href in).
- 3 for Text nodes (e.g., the text content within a <p> tag).
- 9 for Document nodes (the entire HTML document).

nodeName: Returns the name of the node.

- For Element nodes, it's the tag name in uppercase (e.g., "DIV", "P").
- For Text nodes, it's "#text".
- For Document nodes, it's "#document".

nodeValue: Returns or sets the value of the node.

- For Text nodes, it's the actual text content.
- For Attribute nodes, it's the attribute's value.
- For Element nodes, it's null.

parentNode: Returns the parent node of the current node. If there is no parent (e.g., for the Document node), it returns null.

childNodes: Returns a live NodeList of all child nodes of the current node, including text nodes and comment nodes.

firstChild: Returns the first child node of the current node.

lastChild: Returns the last child node of the current node.

nextSibling: Returns the next sibling node in the same parent's childNodes list.

previousSibling: Returns the previous sibling node in the same parent's childNodes list.

Example:

<!DOCTYPE html>

<html>

<body>

```
<h2>JavaScript HTML DOM</h2>
```

```
<p>Add a new HTML Element.</p>
```

```
<div id="div1">
```

```
<p id="p1">This is a paragraph.</p>
```

```
<p id="p2">This is another paragraph.</p>
```

```
</div>
```

```
<script>
```

```
const para = document.createElement("p");
```

```
const node = document.createTextNode("This is new.");
```

```
para.appendChild(node);
```

```
const element = document.getElementById("div1");
```

```
element.appendChild(para);
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT:

JavaScript HTML DOM

Add a new HTML Element.

This is a paragraph.

This is another paragraph.

This is new.

10. Explain the structure of an HTML form. Describe input element types with examples

Ans : An HTML form is used to collect user input. The user input is most often sent to a server for processing. The HTML <form> element is used to create an HTML form for user input:

<form>

.

form elements

.

</form>

The <form> element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

The <input> Element

The HTML <input> element is the most used form element.

An <input> element can be displayed in many ways, depending on the type attribute.

Here are some examples:

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

HTML Input Types

Here are the different input types you can use in HTML:

- <input type="button">
- <input type="checkbox">
- <input type="color">
- <input type="date">
- <input type="datetime-local">
- <input type="email">
- <input type="file">
- <input type="hidden">
- <input type="image">
- <input type="month">
- <input type="number">
- <input type="password">

Input Type Text

`<input type="text">` defines a single-line text input field:

Example

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname">
</form>
```

OUTPUT:

This is how the HTML code above will be displayed in a browser:

First name:

Last name:

Input Type Password

`<input type="password">` defines a **password field**:

Example

```
<form>
  <label for="username">Username:</label><br>
  <input type="text" id="username" name="username"><br>
  <label for="pwd">Password:</label><br>
  <input type="password" id="pwd" name="pwd">
</form>
```

OUTPUT:

This is how the HTML code above will be displayed in a browser:

Username:

Password:

Input Type Submit

`<input type="submit">` defines a button for **submitting** form data to a **form-handler**.

The form-handler is typically a server page with a script for processing input data.

The form-handler is specified in the form's action attribute:

Example

```
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
</form>
```

OUTPUT:

This is how the HTML code above will be displayed in a browser:

First name:

Last name:

11. Explain JSON parsing and serialization in detail. Discuss use cases, examples, and the importance in client-server communication.

Ans :JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is human-readable and easy for machines to parse and generate. JSON is built on two structures: a collection of name/value pairs (object) and an ordered list of values (array).

JSON Parsing (Deserialization):

JSON parsing is the process of converting a JSON-formatted string into a native data structure or object in a specific programming language. When data is received from an external source, such as a web server, it typically arrives as a JSON string. Parsing allows the application to access and manipulate this data using the language's native data types.

1. JSON.stringify()

Imagine we have this object in JavaScript:

```
const obj = {name: "John", age: 30, city: "New York"};
```

Use the JavaScript function JSON.stringify() to convert it into a string.

```
const myJSON = JSON.stringify(obj);
```

The result will be a string following the JSON notation. myJSON is now a string, and ready to be sent to a server:

Example

```
const obj = {name: "John", age: 30, city: "New York"};
const myJSON = JSON.stringify(obj);
```

2. JSON.parse()

Example - Parsing JSON

Imagine we received this text from a web server:

```
'{"name":"John", "age":30, "city":"New York"}'
```

Use the JavaScript function JSON.parse() to convert text into a JavaScript object:

```
const obj = JSON.parse('{"name":"John", "age":30, "city":"New York"}');
```

Example

```
'{"name":"John", "age":30, "car":null}'
```

The example above defines an object with 3 properties:

- "name"
- "age"
- "car"

Each property has a value:

- "John"
- 30
- null

Use Cases:

- **Web APIs:** JSON is the predominant format for data exchange in web services and RESTful APIs.
- **Configuration Files:** Many applications use JSON for storing configuration settings due to its human-readable nature.
- **Data Interchange:** Facilitating data transfer between different systems and programming languages.
- **Client-Server Communication:** Enabling web clients (browsers, mobile apps) to interact with server-side applications.
- **NoSQL Databases:** Databases like MongoDB store data in JSON-like documents, making JSON serialization and parsing crucial.

JSON's role in client-server communication is paramount:

- **Standardization:** Provides a universal, language-agnostic format for data exchange, ensuring interoperability between diverse systems.
- **Readability and Simplicity:** Its human-readable syntax simplifies debugging and development.
- **Efficiency:** While not as compact as binary formats, JSON is generally efficient enough for most web-based data transfer and is easily processed by client-side JavaScript.
- **Flexibility:** Supports complex nested data structures, allowing for rich and detailed data representation.
- **Wide Adoption:** Extensive library support across virtually all programming languages simplifies implementation.

12. Explain scheduler functions: setTimeout() and setInterval() with examples

Ans : The window object allows execution of code at specified time intervals.

These time intervals are called timing events.

The two key methods to use with JavaScript are:

- `setTimeout(function, milliseconds)`
Executes a function, after waiting a specified number of milliseconds.
- `setInterval(function, milliseconds)`
Same as `setTimeout()`, but repeats the execution of the function continuously.

The `setTimeout()` and `setInterval()` are both methods of the HTML DOM Window object.

The setTimeout() Method

`window.setTimeout(function, milliseconds);`

The `window.setTimeout()` method can be written without the window prefix.

The first parameter is a function to be executed.

The second parameter indicates the number of milliseconds before execution.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Timing</h2>

<p>Click "Try it". Wait 3 seconds, and the page will alert "Hello".</p>

<button onclick="setTimeout(myFunction, 3000);">Try it</button>

<script>
function myFunction() {
  alert('Hello');
}
</script>

</body>
</html>
```

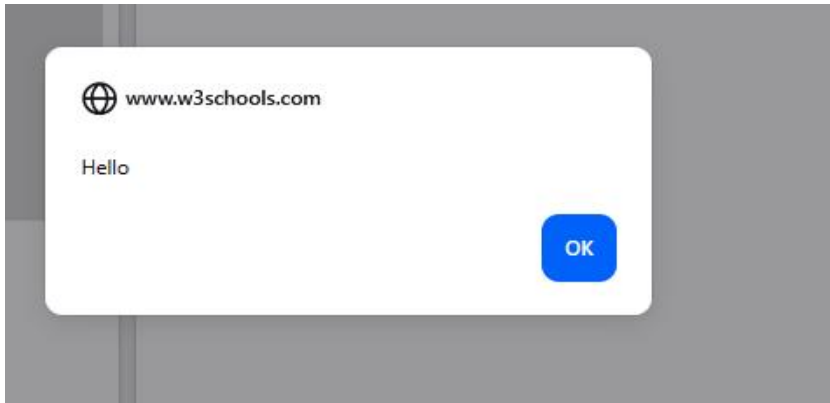
OUTPUT:

JavaScript Timing

Click "Try it". Wait 3 seconds, and the page will alert "Hello".

Try it

After “click”



The setInterval() Method

The setInterval() method repeats a given function at every given time-interval.

`window.setInterval(function, milliseconds);`

The window.setInterval() method can be written without the window prefix.

The first parameter is the function to be executed.

The second parameter indicates the length of the time-interval between each execution.

This example executes a function called "myTimer" once every second (like a digital watch).

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript Timing</h2>
```

```
<p>A script on this page starts this clock:</p>
```

```
<p id="demo"></p>
```

```
<script>
setInterval(myTimer, 1000);

function myTimer() {
  const d = new Date();
  document.getElementById("demo").innerHTML = d.toLocaleTimeString();
}
</script>

</body>
</html>
```

OUTPUT:

JavaScript Timing

A script on this page starts this clock:

12:28:25 pm

13. Explain HTTP communication. Describe request methods, status codes, and headers.

Ans : HTTP communication operates as a request-response protocol between a client (e.g., a web browser) and a server. The client sends an HTTP request to the server, and the server responds with an HTTP response.

Request Methods

HTTP request methods, also known as verbs, indicate the desired action to be performed on a resource:

- **GET:** Retrieves data from a specified resource.
- **POST:** Submits data to be processed to a specified resource, often creating a new resource.

- **PUT:** Replaces all current representations of the target resource with the request payload. If the resource does not exist, it may create it.
- **PATCH:** Applies partial modifications to a resource.
- **DELETE:** Deletes the specified resource.
- **HEAD:** Similar to GET, but requests only the headers of a resource, not the body.
- **OPTIONS:** Describes the communication options for the target resource.

Status Codes

HTTP status codes are three-digit numbers returned by the server in the response, indicating the outcome of the request. They are grouped into five classes:

- **1xx Informational:** Request received, continuing process. (e.g., 100 Continue)
- **2xx Success:** The action was successfully received, understood, and accepted. (e.g., 200 OK, 201 Created)
- **3xx Redirection:** Further action needs to be taken to complete the request. (e.g., 301 Moved Permanently, 302 Found)
- **4xx Client Error:** The request contains bad syntax or cannot be fulfilled. (e.g., 400 Bad Request, 404 Not Found)
- **5xx Server Error:** The server failed to fulfill an apparently valid request. (e.g., 500 Internal Server Error, 503 Service Unavailable)

Headers

HTTP headers are key-value pairs that carry metadata about the request or response. They provide information about the client, server, or the data being exchanged.

Common Request Headers:

- **User-Agent:** Identifies the client software.
- **Accept:** Specifies media types the client can handle.
- **Content-Type:** Indicates the media type of the request body.
- **Authorization:** Carries authentication credentials.

Common Response Headers:

- **Content-Type:** Indicates the media type of the response body.
- **Content-Length:** The size of the response body in bytes.
- **Cache-Control:** Directives for caching mechanisms.

- Set-Cookie: Sends cookies from the server to the client.

14. Describe the JavaScript event model with a neat diagram.

Ans : The JavaScript event model describes how events (user actions or browser actions) are handled and propagated within the Document Object Model (DOM). It involves three main phases: Capturing, Target, and Bubbling.

1. Capturing Phase:

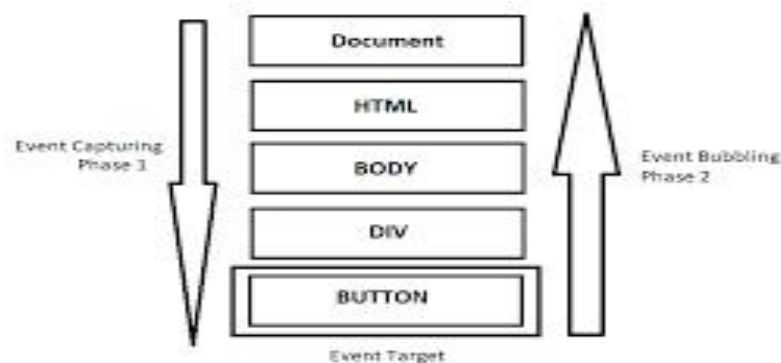
The event starts at the window object and travels down the DOM tree to the target element. During this phase, event listeners registered for the capturing phase on parent elements will be triggered.

2. Target Phase:

The event reaches the target element where the event originally occurred. Event listeners registered directly on this element for either capturing or bubbling will be executed.

3. Bubbling Phase:

The event then travels back up the DOM tree from the target element to the window object. During this phase, event listeners registered for the bubbling phase on parent elements will be triggered.



Here is a list of some common HTML events:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

15. Explain the structure of an HTTP request and HTTP response with diagrams.

Ans : HTTP requests and responses are fundamental to web communication, following a distinct structure for efficient information exchange between clients and servers.

HTTP Request Structure

An HTTP request consists of three main parts:

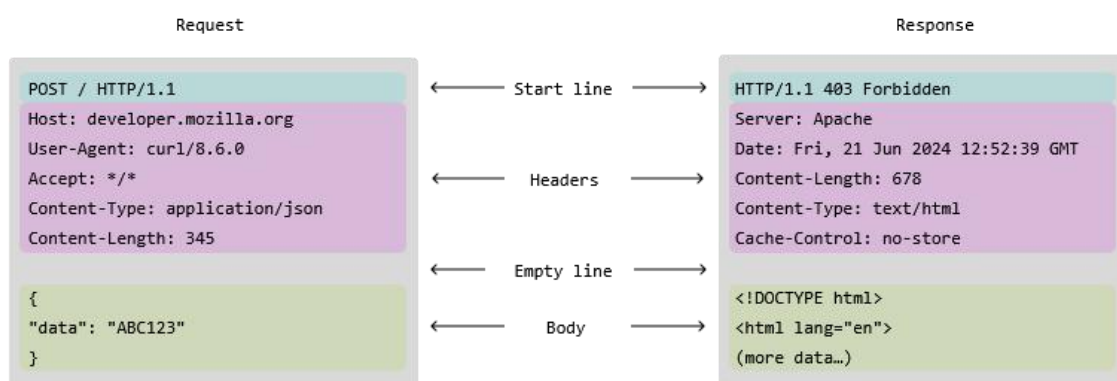
Request Line: This is the first line of the request and contains:

- **Method:** The HTTP method (e.g., GET, POST, PUT, DELETE) indicating the action to be performed.
- **Request-URI:** The Uniform Resource Identifier (URI) identifying the target resource.
- **HTTP Version:** The version of the HTTP protocol being used (e.g., HTTP/1.1).

Request Headers: These lines follow the request line and provide additional information about the request, the client, or the desired response. Examples include:

- **Host:** The domain name of the server.
- **User-Agent:** Information about the client (browser and operating system).
- **Accept:** Preferred media types for the response.
- **Content-Type:** The media type of the request body (if present).

Request Body (Optional): This section contains data being sent to the server, primarily used with methods like POST or PUT for submitting form data or uploading files. It is separated from the headers by an empty line.



HTTP Response Structure

An HTTP response also consists of three main parts:

- **Status Line:** The first line of the response, containing:
 - **HTTP Version:** The version of the HTTP protocol used by the server.
 - **Status Code:** A three-digit code indicating the outcome of the request (e.g., 200 OK, 404 Not Found, 500 Internal Server Error).
 - **Reason Phrase:** A brief, human-readable description of the status code.

Response Headers: These lines follow the status line and provide information about the response, the server, or the content being sent. Examples include:

- Content-Type: The media type of the response body.
- Content-Length: The size of the response body in bytes.
- Server: Information about the server software.
- Date: The date and time the response was generated.

Response Body (Optional): This section contains the actual data or content requested by the client (e.g., HTML, images, JSON data). It is separated from the headers by an empty line.

